

ゼロから始めるScala体験会 in 大阪

2025年5月20日（火）

株式会社ネクストビート

本日のゴールと対象者

- **ターゲット:**
 - Scalaに興味がある方
 - プログラミング経験はあるが、Scalaは初心者レベル
- **ゴール:** 1時間30分で...
 - Scalaの魅力（特に **型安全性**, **表現力**, **関数型** の考え方）の一端を体験
 - `opaque type` や `Either`、関数合成を使って **堅牢なコード** を書くメリットを体感してもらう
 - `given` / `using` を使った型クラスの作り方を体験してもらう

ツールと進め方

- ツール: **Scastie** (ブラウザで動く Scala環境)
 - URL: <https://scastie.scala-lang.org/>
 - デフォルトでScala 3モードで動きます
- 形式:
 - 説明とデモが中心
 - ときどき、Scastieで簡単なコードを試す **演習時間** を設けます

時間配分（目安）

- Scalaの紹介と導入（10分）：18:40-18:50
- 値オブジェクト：case class VS. opaque type（10分）：18:50-19:00
- Eitherを使った値オブジェクトのバリデーション（10分）：19:00-19:10
- 値オブジェクトの型安全な生成（10分）：19:10-19:20
- 休憩（10分）：19:20-19:30
- given/usingを使った型クラスの作り方（15分）：19:30-19:45
- まとめ・プロダクションコード・質疑応答（15分）：19:45-20:00
- 20:00～懇親会（🍷🥂）

※Scastieの使い方とScalaの基本的な構文は[補足資料](#)をご覧ください。

Scalaの紹介と導入

Scalaとは？ (1/2)

- JVM (Java Virtual Machine) 上で動作する **静的型付け** 言語
- Javaとの高い **相互運用性**
 - Java のライブラリをそのまま利用できる
- **オブジェクト指向 + 関数型プログラミング** の融合
 - 両方の良いところを活用できる！
- 強力な **型システム**
 - コンパイル時にエラーを発見しやすく、堅牢なコードを書ける
 - 今日、このメリットを体験します！

Scalaとは？ (2/2)

- **今日の焦点:**
 - Scalaの **型安全性**
 - **関数型プログラミング** の考え方の一部
- **体験するメリット:**
 - **堅牢性:** バグりにくいコード
 - **表現力:** やりたいことを明確にコードで表現できる
 - **組み立てやすさ:** 部品を組み合わせて安全なプログラムを作る

Hello, Scala! (Scastieで試そう！)

まずは定番の "Hello, World!"

```
// このコードをScastieに貼り付けて実行してみましょう!  
println("Hello, World!")
```


ちょっとScalaらしいコード

フィボナッチ数列を生成する例 (雰囲気を感じただけでOK)

```
val fibs: LazyList[BigInt] = {  
  BigInt(0) #:: BigInt(1) #:: fibs.zip(fibs.tail).map { case (a, b) =>  
    a + b  
  }  
}  
println(fibs.take(10).toList)  
// 出力: List(0, 1, 1, 2, 3, 5, 8, 13, 21, 34)
```

- `LazyList`: 必要になるまで計算しないリスト
- `.map`: 関数型らしいデータ変換

値オブジェクト：case class VS. opaque type

値オブジェクト：メールアドレスを「型」で表す

問題: メールアドレスをただの `String` で扱うと？

```
def sendEmail(email: String, subject: String): Unit = ???  
  
// こんな呼び出し方ができてしまう...  
sendEmail("これはメールアドレスじゃない", "テスト")  
sendEmail("", "件名") // 空文字列も渡せる  
sendEmail("user@domain.com", "user-name") // 引数の順番ミスにも気づきにくい
```

課題:

- 不正な値（空文字列、形式が違う文字列）を 代入できてしまう
- 引数が `String` だと、何を表す文字列なのか分かりにくい

解決策: メールアドレス専用の 型 を作る！ => 値オブジェクト

方法1: `case class` による値オブジェクトの表現

`case class` を使うと、簡単に独自の型を定義できます。

```
// Emailという名前の case class を定義
case class Email(value: String)
// Email型として値を保持
val email: Email = Email("test@example.com")
println(email)           // 出力: Email(test@example.com)
println(email.value)     // 出力: test@example.com
// 型が違うので、Stringを直接代入できない! (コンパイルエラー)
// val wrong: Email = "test@example.com"
```

- メリット: 簡単、シンプル
- デメリット: 実行時に `Email` オブジェクト生成のオーバーヘッド

方法2: `opaque type` による値オブジェクトの表現

実行時オーバーヘッドなしで型安全性を実現！(Scala 3 の機能)

- `opaque type Email = String`
 - `Email` 型を作るが、実行時は `String` として扱われる
 - 定義したスコープ内では `Email` と `String` は同じ型として扱われる

```
object Email {  
  // opaque type を定義 (実体は String)  
  opaque type Email = String  
  // ファクトリメソッド (同じスコープであれば、String と Email は同じ扱い)  
  def from(value: String): Email = value  
  // 拡張メソッド (内部の値へのアクセス用)  
  extension (self: Email) def value: String = self  
}
```

opaque type の特徴と比較

- メリット:
 - 実行時オーバーヘッドゼロ！
 - コンパイル時に `String` と `Email` の混同を防げる（型安全性）
- デメリット:
 - `case class` より少し記述が増える

```
import Email.*
val e: Email = Email.from("test@example.com")
println(e) // 出力: test@example.com
println(e.getClass) // 出力: class java.lang.String (実行時は String として扱われる)
// 直接 String は代入できない!(コンパイルエラー)
// val eBad: Email = "test@example.com"
```

値オブジェクトの型安全な生成

値オブジェクトの型安全な生成

opaque type によって、`Email` 型を作ることができました。

しかし、`Email` の値を不正な文字列 (例: `""`, `"not-email"`) から生成することができてしまいます。

```
val invalidE1: Email = Email.from("") // 空文字列を渡してもコンパイルエラーにならない
val invalidE2: Email = Email.from("not-email") // 不正なメールアドレスも渡せてしまう
println(invalidE1) // 出力: ""
println(invalidE2) // 出力: "not-email"
```


Eitherを使った値オブジェクトのバリデーション

目標: `opaque type Email` を、不正な文字列 (例: `""`, `"not-email"`) からは生成できないようにしたい。

アイデア:

1. `Email` を生成する前に、入力文字列を **バリデーション** する
2. バリデーションの結果を `Either` で返すようにする
 - 成功: `Right(検証済み文字列)`
 - 失敗: `Left(エラーメッセージ)`
3. 成功時に `opaque type Email` として値を返す **安全なファクトリメソッド** を作る。

成功 or 失敗 を表す型: `Either`

結果が **成功** か **失敗** のどちらか一方であることを **型** で表現します。

- `Either[L, R]` という型
 - `L`: Left (左側) の型。慣習的に **失敗** 時の情報を入れる
 - `R`: Right (右側) の型。慣習的に **成功** 時の値を入れる

```
// 成功例: Right を使う。Int型の値 100 を持つ
val success: Either[String, Int] = Right(100)
// 失敗例: Left を使う。String型の "エラー発生" を持つ
val failure: Either[String, Int] = Left("エラー発生")
println(success) // 出力: Right(100)
println(failure) // 出力: Left(エラー発生)
```

バリデーション関数の準備

今回はシンプルなチェック関数を自作します。
(実務ではバリデーションライブラリを使うことが多いです)

- 作るチェック関数:

1. `nonEmpty`: 空文字列 ("" や " ") でないか?
2. `containsAtMark`: @ マークを含んでいるか?

- シグネチャ: `String => Either[String, String]`

- 入力: `String`
- 出力: `Either[エラーメッセージ, 検証済み文字列]`

演習：バリデーション関数 (10分)

- [Scastieへのリンク](#)

```
// object Email { ... の中に追加

// 空文字列 (``""` や ``" "` ) でなければ Right、空文字列であれば Left
def nonEmpty(value: String): Either[String, String] = {
  // value.trim で前後の空白を除去してから isEmpty でチェック
  ???
}

// `@` マークを含んでいれば Right、含んでいなければ Left
def containsAtMark(value: String): Either[String, String] = {
  // value.contains でチェック
  ???
}

// }
```

演習：バリデーション関数 (回答例)

```
// object Email { ... の中に追加
def nonEmpty(value: String): Either[String, String] = {
  // value.trim で前後の空白を除去してから isEmpty でチェック
  if (value.trim.isEmpty) {
    Left("Email cannot be empty") // 失敗 -> Left
  } else {
    Right(value) // 成功 -> Right
  }
}

def containsAtMark(value: String): Either[String, String] = {
  if (value.contains('@')) {
    Right(value) // 成功 -> Right
  } else {
    Left("Email must contain '@'") // 失敗 -> Left
  }
}
// }
```

バリデーションの「合成」

`nonEmpty` と `containsAtMark` の両方を順番に適用したい。

- `nonEmpty` で失敗したら、処理を中断して `Left` を返したい。
- `nonEmpty` が成功したら、結果を使って `containsAtMark` を実行したい。

ここで `Either` の関数合成が役立ちます！

とくに `for` 式 (for comprehension) を使うと宣言的に書けます。

安全なファクトリメソッド `from` の実装

`for` 式を使って、複数のバリデーションを **合成** します。

`Either` に対する `for` 式は、途中で `Left` になったら、その後の処理は実行されずに、その `Left` が最終結果となります。

```
// object Email { ... の中に追加

// 複数のバリデーションを合成する関数
def from(value: String): Either[String, Email] = {
  for {
    s1 <- nonEmpty(value)           // 1. nonEmpty を実行。失敗(Left)ならここで終了
    s2 <- containsAtMark(s1)        // 2. s1が成功(Right)の場合のみ実行。失敗ならここで終了
    // 他のチェックもここに追加できる
    // s3 <- checkDomain(s2)
  } yield s2 // 3. 全てのチェックが成功(Right)した場合、最後の結果(s2)が Right で包まれて返る
}

// }
```

全体のコード (Email オブジェクト)

```
object Email {  
  opaque type Email = String  
  
  // --- バリデーション関数 (※privateに変更: Email オブジェクトからのみアクセスできる) ---  
  private def nonEmpty(value: String): Either[String, String] =  
    if (value.trim.isEmpty) Left("Email cannot be empty") else Right(value)  
  
  private def containsAtMark(value: String): Either[String, String] =  
    if (value.contains('@')) Right(value) else Left("Email must contain '@'")  
  
  // --- 安全なファクトリメソッド (バリデーション合成) ---  
  def from(value: String): Either[String, Email] =  
    for {  
      s1 <- nonEmpty(value)  
      s2 <- containsAtMark(s1)  
    } yield s2  
  
  // --- 拡張メソッド ---  
  extension (self: Email) def value: String = self  
}
```


試してみよう！

安全なファクトリメソッド `Email.from` を使ってみましょう。

```
import Email.* // Email オブジェクトの中身を使えるようにする

val validEmailResult    = Email.from("test@example.com")
val emptyEmailResult    = Email.from("")
val noAtMarkResult      = Email.from("testexample.com")
val emptyThenNoAtMark   = Email.from(" ") // 空白のみ

println(s"Valid: $validEmailResult")
println(s"Empty: $emptyEmailResult")
println(s"No '@': $noAtMarkResult")
println(s"Empty then No '@': $emptyThenNoAtMark")
```

実行結果

```
Valid: Right(test@example.com)
Empty: Left>Email cannot be empty)
No '@': Left>Email must contain '@')
Empty then No '@': Left>Email cannot be empty) // 最初の nonEmpty で失敗
```

- 不正な値からは `Left` が返されるようになりました！

値オブジェクトを使った型安全な処理

型安全な関数の定義

`Email.from` で 安全に生成された `Email` 型だけを受け取る関数を定義してみましょう。

```
def sendEmail(email: Email, subject: String): Unit = {  
  // Email.from によってバリデーション済みであることが保証されている!  
  println(s"$email にメールを送信します。件名: $subject")  
}  
  
// こんな呼び出し方はできない!  
// sendEmail("これはメールアドレスじゃない", "テスト")  
// sendEmail("", "件名")  
// sendEmail("user@domain.com", "user-name") // 引数の順番ミスにも気づける
```

型安全な関数を使う

`Email.from` の型は `Either[String, Email]` なので、そのまま渡すことはできません。
`Either` の結果は、`match` 式を使って取り出すことができます。

```
val validEmailResult = Email.from("test@example.com")
val invalidEmailResult = Email.from("invalid-email")

validEmailResult match {
  case Right(emailInstance) => sendEmail(emailInstance, "テスト件名")
  case Left(error)          => println(s"Match Failed: $error")
}
// 出力: test@example.com にメールを送信します。件名: テスト件名

invalidEmailResult match {
  case Right(emailInstance) => sendEmail(emailInstance, "テスト件名")
  case Left(error)          => println(s"Match Failed: $error")
}
// 出力: Match Failed: Email must contain '@'
```

Either の結果を安全に使う (**fold**)

match の代わりに **fold** メソッドもよく使われます。

```
println("\nUsing fold:")

validEmailResult.fold(
  error => println(s"Fold Failed: $error"), // 第1引数: Left の場合の処理
  email => sendEmail(email, "テスト件名")   // 第2引数: Right の場合の処理
)
// 出力: test@example.com にメールを送信します。件名: テスト件名

invalidEmailResult.fold(
  error => println(s"Fold Failed: $error"),
  email => sendEmail(email, "テスト件名") // こちらは実行されない
)
// 出力: Fold Failed: Email must contain '@'
```

まとめ: 体験したこと ✨

- 値オブジェクト: ドメインの概念 (例: Email) を **型** で表現する手法。
 - `case class` (シンプル) vs `opaque type` (実行時コストゼロ!)
- `Either[L, R]`: 成功(**Right**) または 失敗(**Left**) を型レベルで表現。
- 関数合成 (**for** 式): 複数の処理 (バリデーションなど) を宣言的かつ安全に **組み合わせる**。途中で失敗したら中断できる!
- 型安全性:
 - `opaque type` で `String` と `Email` の混同を **コンパイル時に** 防ぐ。
 - 関数の引数型 (`def f(e: Email)`) で意図を明確にし、不正な利用を **コンパイル時に** 防ぐ。
 - `Either` の `match` や `fold` で、成功/失敗の処理漏れを **コンパイル時に** 防ぐ。

given/usingを使った型クラスの作り方

given/usingを使った型クラスの作り方

型クラスは、特定の操作（メソッド）をサポートする型を抽象化するデザインパターンです。Scala 3では `given` と `using` を使って型クラスをより扱いやすく記述できます。

- 型クラスの定義:

- 共通の操作を定義する `trait` を使います。

- 型クラスインスタンスの定義 (`given`):

- 特定の型が型クラスの操作をどのように実装するかを `given` を使って定義します。

- 型クラスの利用 (`using`):

- 型クラスのインスタンスが必要な関数やメソッドの引数に `using` を付けます
- コンパイラが適切な `given` インスタンスを自動で見つけて渡してくれます。

given/usingを使った型クラスの作り方（コード例）

例として、値を文字列に変換する `Show` という型クラスを考えます。

```
// 1. 型クラスの定義 (Show トレイト)
trait Show[A] {
  def show(value: A): String
}

// 2. Int型とString型に対するShowインスタンスの定義 (given)
given intShow: Show[Int] with {
  def show(value: Int): String = value.toString
}

given stringShow: Show[String] with {
  def show(value: String): String = s"$value" // 文字列は引用符で囲む
}

// 3. 型クラスを利用する関数 (using)
def display[A](value: A)(using s: Show[A]): Unit = {
  println(s.show(value))
}

// 4. 型クラスの利用
display(123) // 出力: 123 (Intのgivenインスタンスが使われる)
display("hello") // 出力: "hello" (Stringのgivenインスタンスが使われる)
```

given/usingを使った型クラスの作り方（コード例） - 補足

- givenインスタンスは名前を省略することもあります

```
// 1. 型クラスの定義 (Show トレイト)
trait Show[A] {
  def show(value: A): String
}

// 2. Int型に対するShowインスタンスの定義 (given)
given Show[Int] with {
  def show(value: Int): String = value.toString
}

// 3. String型に対するShowインスタンスの定義 (given)
given Show[String] with {
  def show(value: String): String = s"$value" // 文字列は引用符で囲む
}

...
```

- `given Show[Int]` のようにすれば名前を省略できます。

given/usingを使った新しい型クラスの作り方

- `Boolean` 型に対する `Show` インスタンスを使って定義します。

```
given Show[Boolean] with {  
  def show(value: Boolean): String = value match {  
    case true => "True"  
    case false => "False"  
  }  
}
```

```
display(true) // True  
display(false) // False
```

型クラスの別の例: Monoid

Monoid は、ある型 **A** に対して、以下の2つの条件を満たす操作を定義する型クラスです。

1. **結合律 (Associativity):** `(a combine b) combine c` は `a combine (b combine c)` と等しい。
2. **単位元 (Identity Element):** ある特別な要素 `empty` が存在し、`a combine empty` も `empty combine a` も `a` と等しい。

数値の加算 (`+`) と `0`、文字列の結合 (`++`) と `""`、リストの連結 (`++`) と `Nil` などが Monoid の例です。

演習：Monoid型クラスのインスタンス定義（10分）

- **Monoid** 型クラスに対して、**given** インスタンスを定義してみましょう。
 - [Scastieへのリンク](#)

```
trait Monoid[A] {  
  def combine(x: A, y: A): A // 要素を組み合わせる操作  
  def empty: A // 単位元  
}  
  
given Monoid[Int] with {  
  def combine(x: Int, y: Int): Int = ???  
  def empty: Int = ???  
}  
  
given Monoid[String] with {  
  def combine(x: String, y: String): String = ???  
  def empty: String = ???  
}  
  
given Monoid[List[A]] with {  
  def combine(x: List[A], y: List[A]): List[A] = ???  
  def empty: List[A] = ???  
}
```

演習：Monoid型クラスのインスタンス定義（回答）

```
trait Monoid[A] {  
  def combine(x: A, y: A): A // 要素を組み合わせる操作  
  def empty: A // 単位元  
}  
  
given Monoid[Int] with {  
  def combine(x: Int, y: Int): Int = x + y  
  def empty: Int = 0  
}  
  
given Monoid[String] with {  
  def combine(x: String, y: String): String = x + y  
  def empty: String = ???  
}  
  
given Monoid[List[A]] with {  
  def combine(x: List[A], y: List[A]): List[A] = x ++ y  
  def empty: List[A] = Nil  
}
```

Monoid型クラスの利用とusing

```
// Monoid[A] が利用可能であれば、List[A] を畳み込める関数
def combineAll[A](list: List[A])(using m: Monoid[A]): A = {
  list.foldLeft(m.empty)(m.combine) // foldLeftを使って畳み込む
}

// List[Int]を畳み込む (合計)
val numbers = List(1, 2, 3, 4, 5)
println(s"Sum: ${combineAll(numbers)}") // Sum: 15

// List[String]を畳み込む (結合)
val words = List("Hello", " ", "World", "!")
println(s"Combined words: ${combineAll(words)}") // Hello, World!

// List[Int]のリストを畳み込む (連結)
val listOfLists = List(List(1, 2), List(3), List(4, 5))
println(s"lists: ${combineAll(listOfLists)}")
// 出力: Combined lists: List(1, 2, 3, 4, 5)
```


まとめ: 体験したこと ✨

今日体験した機能はScalaの魅力のほんの一部です。

- **堅牢性:** 型システムや `Either` などが、バグを未然に防ぎ、信頼性の高いコードを書く助けになる。
- **表現力:** `opaque type` や `for` 式のように、プログラマの意図をコードで明確に表現しやすい。
- **組み立てやすさ:** 小さな関数（バリデーション）や型（`Either`）を組み合わせ、安全で複雑な処理を構築できる（関数型プログラミングの考え方）。

皆さんのScalaへの興味を深めるきっかけになれば幸いです！

(任意) 次のステップ

- **Scala公式サイト:**
 - Scala 3 Book: <https://docs.scala-lang.org/scala3/book/introduction.html>
 - Tour of Scala: <https://docs.scala-lang.org/tour/tour-of-scala.html>
- **オンライン学習:**
 - Scala Exercises: <https://www.scala-exercises.org/>
- **Scastieで色々試してみる！**
 - <https://scastie.scala-lang.org/>

実際のコードを見てみましょう

質疑応答